

Interpreter  
if prim  $\rightarrow$  ret value  
if name  $\rightarrow$  ret value  
(var)

if name → ret value  
(var)

if combination  $\rightarrow$  1. eval subexpressions 2. apply operator to args

if prim do it

↳ if lambda eval body  
with args

$$(\mathcal{L}(p)(p^3))$$
$$(number \rightarrow A) \rightarrow A$$

car, cdr, cons, null?, nil, list?, append, contains?

$$A \in \text{LOI} \text{ iff } 1. A = () \text{ OR } 2. A \text{ is a pair w/ } \text{car} \rightarrow \text{int}, \text{cdr} \rightarrow \text{LOI}$$

LOI is the smallest set that satisfies 1.  $() \in \text{LOI}$  2.  $(n.l) \in \text{LOI}$   $\begin{matrix} n \in \text{int} \\ l \in \text{LOI} \end{matrix}$

$$\frac{() \in \text{LoI} \quad n \in \text{Int} \quad \ell \in \text{LoI}}{(n.\ell) \in \text{LoI}} \Rightarrow \frac{\text{antecedents}}{\text{consequents}}$$

interface

<sup>internal</sup> <sup>representation</sup> <sup>interpretation</sup>

|              |                      |                          |                     |
|--------------|----------------------|--------------------------|---------------------|
| define-code, | define-datatype exp, | define-datatype exprval, | define value-of     |
|              | <sup>exp?</sup>      | <sup>exprval?</sup>      | <sup>exp, env</sup> |

representation

interpret

(num-val (value num?))

(procedure var iden?)

(body exp?)

```
(env env?)
```

(define-datatype environment env-?

(empty-enw)

```
(extend-env (var iden?)
```

(val exp-val?)

(enw enu?)

```

+ (extend-env-rec (pname iden?)

```

(pname iden?)

(övar iden?

(pbody exp?)

(env env?)

(apply-env (1 env var))

(cases environment env\_

```
(empty-env () error)
```

```
(extend-env (var1 val1
              env0)
```

(if (eq v? var 1 var) val 1

```
(apply-env env0 var))
```

+ (extend-env-rec (name  
p var  
p body)  $\rho_0$ )

```
(if (epv? var pname)
```

(proc-val (procedure

prave, bvar pbody

(apply-env env0 var)))

(let-rec-exp pname bvar pbody : letrec)

valuing (letrec-body (extend-env) ... pbody f)

store

(define (gcd a b) (if (= (remainder a b) 0) b (gcd b (remainder a b))))

Iterative - no pending operations  $acc = 0$   
for ((i = -; i < K; i++))

## Trees

(( (1 2) 3) (4 (5 6)) 7 (8 9 10))

(define count-leaves (lambda (tree)  
cond ((not (list? tree)) 1)  
((null? tree) 0)  
(else (+ (c-l (car tree)) (c-l (cdr tree)))))

filter map fold-right (reduce)  
lst proc lst proc accumulator lst

(define (transpose m) (if (or (null? m) (null? (car m))) ()  
(cons (map car m) (transpose (map cdr m)))))

## Quote

(+ '1 '2) → 3 "a" → 'a (eq? 'a 'a) → #t

## Lexical Addr

let b = 1  
let a = -3  
let f = proc (a) (proc (b) (exp...)) in  
call exp

implicit index depth  
#0 #1 ⇒ # contours to cross to reach declaration  
#0 indexed

nameless translation  
let 1  
in let -3  
in let proc  
proc  
- (#2, #3)  
in ((#0 #1) #2)

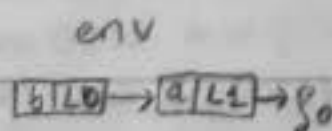
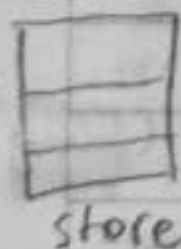
static-env ⇒ [b | a | f] input: var, send  
output: #

nameless-env ⇒ [a<sub>0</sub> | a<sub>1</sub> | a<sub>2</sub> | a<sub>3</sub>] input: #,  
output: value

apply-senv: var = car(senv)? 0; (1 + apply-senv var, cdr(senv))  
(var, senv)

no need to store env state in proc

don't add values, just changes them to value in translation  
when interpreting translation, add values when let exp, call exp.



EREF ⇒ expval: int, bool, proc, ref (ExpVal) newref  
deref setref

IREF ⇒ denvval: ref → also a new type  
expval: int, bool, proc let  
set (assign exp var exp)

let f = 0  
in begin  
set f = proc.. (recursive)  
(f 3)  
end

let-exp, call-exp now  
store values.

MUTPAIR (Ref(ExpVal), Ref(ExpVal))

pair  
left  
right  
setleft  
setright

## Review

Formal parameter:  $\text{proc}(x)$

Actual parameter (operand):  $f(x, 1)$

Denoted value: when called, formal param is bound to the den value.

Proc/Letrec (Natural param passing)

Den value = expressed value  
formal param of actual param

$(\text{valueof } (\text{var-exp var}) g) = g(\text{var})$

$(\text{apply-proc } (\text{proc var body } g_0) \text{ val}) = (\text{value-of body } [\text{var}=\text{val}] g_0)$

$(\text{valueof rator } g) = \text{procval}$

$(\text{expval} \rightarrow \text{proc procval}) = \text{proc}$

$(\text{valueof rand } g) = \text{arg}$

$(\text{apply-proc proc arg}) = \text{val}$

$(\text{valueof } (\text{call-exp rator rand}) g)$

IREF(CBV):

Den value = ref to a loc containing  
formal param = expressed val of actual-param

$\text{expval} = \text{int} + \text{bool} + \text{proc}$

$\text{denval} = \text{ref}(\text{expval})$

$(\text{valueof } (\text{var-exp var}) g \sigma) = (\sigma(g(\text{var})), \sigma)$

$(\text{apply-proc } (\text{proc var body } g_0) \text{ val } \sigma_0) =$

$(\text{value-of body } [\text{var}=\text{loc}] g_0 [\text{loc}=\text{val}] \sigma_0) = (\text{val}, \sigma_1)$

$(\text{valueof rator } g \sigma_0) = (\text{procval}, \sigma_1)$

$(\text{expval} \rightarrow \text{proc procval}) = \text{proc}$

$(\text{valueof rand } g \sigma_1) = (\text{arg}, \sigma_2)$

$(\text{apply-proc proc arg } \sigma_2) = (\text{val}, \sigma_3)$

$(\text{valueof } (\text{call-exp ...}) g \sigma)$

everything in CBR  
returns expval,  
except value of  
operand

CBR:

Den value = existing ref if operand is var

formal param = new location containing exp val of actual param  
othw

$(\text{valueof } (\text{var-exp var}) g \sigma) = \text{same ref}$

$\text{ref}(\text{valueof-operand } (\text{var-exp var}) g \sigma) = (g(\text{var}), \sigma)$

$(\text{valueof-operand exp } g \sigma_0) \Rightarrow (\text{valueof exp } g \sigma_0) = (\text{val}, \sigma_1)$   
 $= (l, [l=\text{val}] \sigma_1)$

$(\text{apply-proc } (\text{proc val body } \rho_0) \text{ ref } \sigma_0) =$   
 $(\text{valueof body } [\text{var} \mapsto \text{ref}] \rho_0 \sigma_0) = (\text{val}, \sigma_1)$

$(\text{value of rator } \rho \sigma_0) = (\text{procval}, \sigma_1)$   
 $(\text{expval} \rightarrow \text{proc procval}) = \text{proc}$   
 $(\text{valueof-operand rand } \rho \sigma_1) = (\text{ref}, \sigma_2)$   
 $(\text{apply-proc proc ref } \rho \sigma_2) = (\text{val}, \sigma_3)$

$(\text{valueof } (\text{call-exp } \dots) \rho \sigma_0)$

Lazy evaluation:

$\text{den val} = \text{existing ref if operand is var}$   
 $\text{formal param} \quad \text{thunk otherwise}$

$(\text{define-datatype thunk thunk?})$   
 $(\text{a-thunk } (\text{exp } \perp \text{ expression?})$   
 $(\text{env environment}))$

$\text{expval} = \text{int, bool, proc}$   
 $\text{denval} = \text{ref}(\text{expval} + \text{thunk})$

$(\text{valueof } (\text{call-exp rator rand}) \rho \sigma_0) = \text{same as CBR}$   
 $(\text{valueof } (\text{var-exp var}) \rho \sigma) = (\sigma(\rho(\text{var})), \sigma)$   $\rightarrow$  if expval  
 $\rightarrow$  if thunk  
 $\text{set ref if CBR} \Rightarrow (\text{valueof-thunk } (\sigma(\rho(\text{var})), \sigma))$   
 $(\text{valueof-thunk } (\text{thunk exp } \rho_0 \sigma) \sigma) = (\text{valueof exp } \rho_0 \sigma)$   
 $(\text{valueof-operand } (\text{var-exp var}) \rho \sigma) = (\rho(\text{var}), \sigma)$   
 $(\text{valueof-operand exp } \rho \sigma) = (t, [t = (\text{a-thunk-exp } \rho)] \sigma)$

- Call by name

evaluate when needed but do not store result

- Call by need

evaluate when needed and store the result

Note: env is packaged but store is changed  
this enables lazy streams (infinite codes)

Thunk is created only when call-exp

$+ \langle \text{expval}, \text{env} \rangle$   
 $(\text{expval}, \text{env})$